

长度平方增长，也就是 $O(n^2)$ 。

8.1.11 Python 交互实验室

正文里我们用手算走完了注意力的三步操作，但手算只能处理两三个词的迷你句子。配套资源包中的 `attention_intuition_demo.py` 把这套计算完整地实现了出来，并且会自动画出注意力权重的柱状图和向量分叉的二维坐标图。这里的脚本是一个“点积近似版”补充实验，会用向量点积重新生成打分，而不是完全复用正文里的预设分数表；它的目的不是提前讲完 QKV，而是让你看到“相似度变化会改变注意力权重”这件事。运行之后，你会看到“苹果”在两个语境下的注意力分布和最终向量被并排放在一起。

最值得尝试的一件事：打开代码顶部的参数配置区，把“吃”的向量从 $[1, 2]$ 改成 $[0, 1]$ ，让它和“苹果”不那么相似，然后重新运行，观察“吃”的注意力权重下降了多少、“苹果”的新向量又偏向于哪个方向。你还可以试着自己编一个全新的句子（比如三个词、每个词两维），看看注意力会把目标词的向量拉向什么位置。改几轮参数之后，“打分越高的词对新向量影响越大”这件事就会从一个你知道的结论，变成一个你看到过的事实。

8.2 Q、K、V：注意力分数到底是怎么算出来的

8.2.1 分数到底是怎么来的

“我喜欢吃苹果”这句话，经过注意力机制之后，“苹果”给“吃”打了最高分，“吃”的信息被大量灌入“苹果”的新向量里。整个过程很清楚：打分、Softmax 归一化、加权平均。但有一个关键环节被跳过了：前面那张 4×4 注意力分数表里的数字，到底是怎么算出来的？

前面给出的分数是直接列在表里的，当时说“先不纠结这些分数是怎么算出来的”。现在是时候把这个黑箱打开了。

要给两个词打一个分，衡量它们之间的相关程度，我们需要一种运算，能接受两个向量作为输入，输出一个数字。这个数字越大，代表两个向量越匹配。学线性代数的时候你见过一个运算恰好能做这件事：**点积**。

点积的计算非常简单：两个向量对应位置的数字相乘，然后把所有乘积加起来，得到一个数字。比如 $[1, 2]$ 和 $[3, 4]$ 的点积就是 $1 \times 3 + 2 \times 4 = 3 + 8 = 11$ 。再比如 $[1, 0]$ 和 $[0, 1]$ 的点积是 $1 \times 0 + 0 \times 1 = 0$ 。

点积有一个很好的性质：两个向量的方向越一致，点积越大；方向垂直，点积为零；方向相反，点积为负。

那最直接的想法就是：拿两个词的向量直接算点积，把结果当作它们之间的注意力分数。我们来试试看，用之前那个例子里的向量算一算，看看效果怎么样。

“我喜欢吃苹果”，四个词的 Embedding 向量（2 维，和前面用的一样）：

词	向量
我	[0, 1]
喜欢	[1, 0]
吃	[1, 2]
苹果	[1, 1]

现在我们用点积给每个词对每个词打分。四个词对四个词，一共 16 个点积，排成一张表：

	我	喜欢	吃	苹果
我 [0, 1]	1	0	2	1
喜欢 [1, 0]	0	1	1	1
吃 [1, 2]	2	1	5	3
苹果 [1, 1]	1	1	3	2

“苹果”对“吃”的分数： $[1, 1]$ 和 $[1, 2]$ 的点积 $= 1 \times 1 + 1 \times 2 = 3$ 。“吃”对“吃”自己的分数： $[1, 2]$ 和 $[1, 2]$ 的点积 $= 1 \times 1 + 2 \times 2 = 5$ 。“我”对“喜欢”的分数： $[0, 1]$ 和 $[1, 0]$ 的点积 $= 0 \times 1 + 1 \times 0 = 0$ 。

看最后一行，也就是“苹果”的视角。它给“吃”打了 3 分，是四个里面最高的，这和直觉一致。但看第一行，也就是“我”的视角。“我”给“喜欢”打了 0 分。零分。在“我喜欢吃苹果”这句话里，“我”和“喜欢”是一个紧密的主谓结构，应该高度相关，结果注意力分数是 0。为什么？因为 $[0, 1]$ 和 $[1, 0]$ 恰好垂直：一个纯在第二维，一个纯在第一维，点积为零。不是因为它们语义上不相关，而是因为 Embedding 碰巧把它们的向量放在了互相垂直的方向上。

再看第三行，也就是“吃”的视角。“吃”给自己打了 5 分，比给“苹果”的 3 分高出一大截。这意味着“吃”最关注的是它自己，而不是它的宾语“苹果”。这也不太合理：对于“吃”来说，最重要的上下文信息应该是“吃什么”，也就是“苹果”。

问题出在哪里？

8.2.2 一个向量身兼数职的困境

问题的根源是：每个词只有一个向量，而这一个向量同时在做两件事。

当“苹果”去给其他词打分的时候，它用的是自己的向量 $[1, 1]$ 。当“苹果”被别的词打分的时候，别人看到的也是这同一个向量 $[1, 1]$ 。也就是说，“苹果”用来“提问”的信息和用来“被查询”的信息，是同一组数字。

可是这两件事不是一回事。“苹果”在“提问”的时候，它想找的是“谁决定了我的含义”，答案是“吃”这个动词。但“苹果”在“被查询”的时候，比如“吃”来找它打分，“苹果”应该展现的是“我是一种食物”这个属性。“想找什么”和“能提供什么”是两个不同的需求，但点积打分迫使它们用同一个向量来表达。

结果就是，打分完全取决于原始 Embedding 向量碰巧指向什么方向。“我”和“喜欢”的向量恰好垂直，分数就是 0。问题不在它们是否相关，而在于 Embedding 没有专门为“打注意力分数”这件事优化过。

Transformer 的做法是：不要直接拿原始向量去打分。先把原始向量做一次变换，变成专门用来打分的新向量，然后再算点积。

8.2.3 乘一个矩阵，换一副面孔

什么样的变换？矩阵乘法。

如果你拿一个向量乘以一个矩阵，出来的是一个新向量。本节为了让手算和后面矩阵形式一致，先按行向量写：向量放在左边，矩阵放在右边，也就是 xW 。前面线性代数里见过的 Wx 是另一套写法，本质都是用矩阵做线性变换。两种写法本质上是转置关系，选哪一种都能讲清楚；关键是同一段计算里保持一个约定，不要一会儿左乘、一会儿右乘。新向量的每个分量都是原始向量各分量的加权组合，而具体怎么组合，由矩阵里的数字决定。这个操作在深度学习里常被叫做投影（projection）。严格来说，它就是一次线性变换：用矩阵把向量从一个空间搬到另一个空间，搬的过程中各个分量会被重新组合。你可以想象成给同一个人换一套衣服：人还是那个人，但外观变了，在不同场合下的辨识度也不一样了。后面我会直接说“投影”来指这个“向量乘以矩阵得到新向量”的操作。

Transformer 的做法是：准备三个不同的矩阵，让每个词的原始向量分别乘以这三个矩阵，得到三个不同的新向量。三个矩阵各负责一件事。先把计算做完，再回头看它们分别在干什么。它们之后会分别叫 Query、Key、Value；名字和作用算完马上揭晓。

三个矩阵如下（ 2×2 ，数字是我编的，目的是让计算过程清晰）：

$$W_1 = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \quad W_2 = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \quad W_3 = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}$$

这里的矩阵是 2×2 的，输入 2 维，输出也是 2 维。但这不是必须的，投影矩阵完全可以把一个高维向量压缩到低维。比如，在真实的大模型中，每个词的向量维度可能有几千维（以一个 7B 模型为例通常是 4096 维，这个维度一般记作 d_{model} ），但投影之后的向量可能只有 128 维。唯一的硬约束是：第一个矩阵和第二个矩阵投影出来的向量维度必须相同，因为它们后面要做点积，点积要求两个向量一样长。第三个矩阵投影出来的维度理论上可以不同，虽然实际中通常也设成一样。至于为什么要往低维压缩，这和后面要讲的多头注意力有关，到时候会说。

我们拿“苹果”的原始向量 $[1, 1]$ 来做一遍。

乘以第一个矩阵 W_1 ：

$$[1, 1] \times W_1 = [1 \times 1 + 1 \times 0, 1 \times 1 + 1 \times 1] = [1, 2]$$

乘以第二个矩阵 W_2 ：

$$[1, 1] \times W_2 = [1 \times 1 + 1 \times 1, 1 \times 0 + 1 \times 1] = [2, 1]$$

乘以第三个矩阵 W_3 ：

$$[1, 1] \times W_3 = [1 \times 1 + 1 \times 0, 1 \times (-1) + 1 \times 1] = [1, 0]$$

同一个原始向量 $[1, 1]$ ，乘以三个不同的矩阵，变成了三个完全不同的向量： $[1, 2]$ 、 $[2, 1]$ 、 $[1, 0]$ 。矩阵里的数字不同，重新组合的方式就不同，出来的新向量也不同。

现在我把四个词都做同样的处理，把结果列成一张表。每个词的原始向量分别乘以三个矩阵，得到三列新向量（计算方法和上面“苹果”的例子完全一样，你可以自己验一两个）：

词	原始向量	$\times W_1$ 的结果	$\times W_2$ 的结果	$\times W_3$ 的结果
我	[0, 1]	[0, 1]	[1, 1]	[0, 1]
喜欢	[1, 0]	[1, 1]	[1, 0]	[1, -1]
吃	[1, 2]	[1, 3]	[3, 2]	[1, 1]
苹果	[1, 1]	[1, 2]	[2, 1]	[1, 0]

现在可以告诉你这三个矩阵分别在干什么了。

第一个矩阵 W_1 产生的向量，叫做 Query（查询）。它代表这个词“在找什么信息”。你可以理解为：这个词向整句话发出的一个“问题”。

第二个矩阵 W_2 产生的向量，叫做 Key（键）。它代表这个词“能提供什么信息”。你可以理解为：这个词挂出来的一块牌子，告诉别人“我有这些东西”。

第三个矩阵 W_3 产生的向量，叫做 Value（值）。它代表“如果你选中了我，我实际给你的内容”。

这三个矩阵一般记作 W_Q 、 W_K 、 W_V 。Query、Key、Value 这组名字借自数据库：在数据库里，你用一个查询去匹配一堆键，命中之后取出对应的值。类比大体成立，所以这组名字就这么流传下来了。在中文技术社区里，几乎没人说“查询矩阵”“键矩阵”“值矩阵”，大家都是直接说 QKV，三个字母连读，已经变成一个固定词组了。你也这么用就好。它们是模型的参数，和前面讲过的线性层权重是同一种东西，在训练开始时随机初始化，然后通过反向传播一点一点调整，最终学会“什么样的查询应该匹配什么样的键”。没有人手动设计这些矩阵的值。

有一件事要在这里说清楚：在一句话内部，Query、Key、Value 这三组向量全都来自同一组原始向量。不是说 Query 来自一个地方、Key 来自另一个地方。它们就是同一个词的同个原始向量，分别乘以三个不同的矩阵，生出来的三个不同的投影。三个角色，但同一个来源。

我们把表格更新一下名字：

词	原始向量	Query (q)	Key (k)	Value (v)
我	[0, 1]	[0, 1]	[1, 1]	[0, 1]
喜欢	[1, 0]	[1, 1]	[1, 0]	[1, -1]
吃	[1, 2]	[1, 3]	[3, 2]	[1, 1]
苹果	[1, 1]	[1, 2]	[2, 1]	[1, 0]

实战避坑 · QKV 的偏置项：有的模型有，有的模型没有

W_Q 、 W_K 、 W_V 的基础形式只展示矩阵乘法，真实实现还要注意偏置项 (bias)。在实际代码里，线性层通常默认带偏置，比如 PyTorch 的 `nn.Linear` 默认 `bias=True`。问题是，不同模型对这个选择不一样：早期的 BERT 类模型在注意力层使用偏置项，而现在主流的 LLaMA、Qwen 等模型则去掉了偏置。如果你要加载一个预训练模型的权重，你的代码里写了 `bias=True`，但模型权重里根本没有 `bias` 这一项，加载的时候会直接报 `key mismatch` 的错。反过来也一样。更隐蔽的情况是：你把别人的注意力实现复制过来用，没注意到原作者的代码是按带 `bias` 写的，而你要适配的模型不带 `bias`，结果多出来的参数被随机初始化了，微调的时候效果莫名其妙地差。养成习惯：拿到一个新模型，先看一眼 config 文件里 `attention_bias` 或 `qkv_bias` 这类字段，确认清楚再写代码。

8.2.4 用 Query 和 Key 重新打分

现在我们拿着这些投影后的向量，重新给“苹果”打一次分。

规则是：拿“苹果”的 Query，去和每个词的 Key 做点积。

苹果的 Query 是 $[1, 2]$ 。

- 和“我”的 Key $[1, 1]$ 做点积： $1 \times 1 + 2 \times 1 = 3$
- 和“喜欢”的 Key $[1, 0]$ 做点积： $1 \times 1 + 2 \times 0 = 1$
- 和“吃”的 Key $[3, 2]$ 做点积： $1 \times 3 + 2 \times 2 = 7$
- 和“苹果”自己的 Key $[2, 1]$ 做点积： $1 \times 2 + 2 \times 1 = 4$

QK 分数： $[3, 1, 7, 4]$

和前面直接用原始向量算点积得到的分数 $[1, 1, 3, 2]$ 对比一下。两次“吃”都是最高分，但差距完全不同：直接点积时，“吃”只比“苹果”自己高了 1 分（3 比 2）；QKV 投影之后，“吃”比第二名高了 3 分（7 比 4）。投影让“吃”的优势从微弱领先变成了遥遥领先。图 8.4 把两种打分方式放在一起，这个差距一目了然。

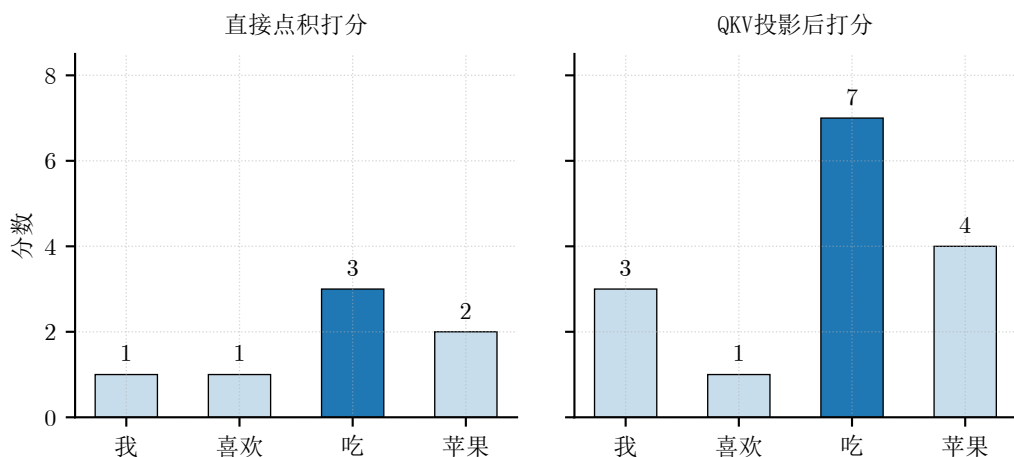


图 8.4：“苹果”在两种打分方式下的注意力分数对比。左图使用原始 Embedding 向量直接做点积，“吃”仅以微弱优势领先；右图使用 QKV 投影后的向量打分，“吃”的优势被显著拉大。深色柱为“吃”，浅色柱为其他词。

你可能还惦记着前面提到的另外两个问题：“我”给“喜欢”打了 0 分、“吃”给自己打了最高分。换成 QKV 之后，这两个问题有没有好转？如果你拿我这组矩阵去算，会发现它们并没有改善：“我”的 Query $[0, 1]$ 和“喜欢”的 Key $[1, 0]$ 做点积还是 0。这很正常，因为我这三个矩阵不是训练出来的，是随手编的整数，纯粹为了让你看清计算过程。真实模型里的 W_Q 和 W_K 是在海量文本上经过无数轮反向传播精心调整出来的，它学到的矩阵会让该高分的词对确实拿到高分。我们在这里能看到的是**机制本身**：三个矩阵给了模型一个可以调节打分方式的旋钮，而这个旋钮最终能调到什么程度，靠的是训练。

面试考点 · 注意力分数矩阵是对称的吗？

常见问法：“在自注意力中，词 A 对词 B 的注意力分数，和词 B 对词 A 的注意力分数，一定相等吗？为什么？”

典型错误：“两个词互相打分，结果当然一样吧？都是算点积。”这是最常见的错误直觉。

正确思路：不相等。A 对 B 的分数是 $q_A \cdot k_B$ ，B 对 A 的分数是 $q_B \cdot k_A$ 。因为 Q 和 K 是用不同的矩阵投影出来的， q_A 和 k_A 是两个完全不同的向量。所以 $q_A \cdot k_B$ 和 $q_B \cdot k_A$ 没有理由相等。用这个具体数字可以直接验证：苹果对吃的分数是苹果的 Query [1, 2] 点积吃的 Key [3, 2] = 7，但吃对苹果的分数是吃的 Query [1, 3] 点积苹果的 Key [2, 1] = 5。 QK^T 矩阵不是对称矩阵。这意味着注意力关系是有方向性的：“我关注你”和“你关注我”的程度可以完全不同。

8.2.5 完整的计算流程：从打分到输出

拿到分数之后，后面的流程和前面讲注意力直觉时完全一样：缩放、Softmax 归一化、对 Value 加权求和。但这里多了一个“缩放”步骤，是前面那个朴素版里没有出现过的。先把它做完，下一小节单独解释为什么要做。

第一步，缩放：把每个分数除以 $\sqrt{d_k}$ 。

d_k 是 Query 和 Key 向量的维度。我们这个例子里 $d_k = 2$ ，所以 $\sqrt{d_k} = \sqrt{2} \approx 1.41$ 。
缩放后的分数：

$$[3 \div 1.41, 1 \div 1.41, 7 \div 1.41, 4 \div 1.41] \approx [2.13, 0.71, 4.96, 2.84]$$

第二步，Softmax 归一化。

对缩放后的四个分数做 Softmax：

- $e^{2.13} \approx 8.41$
- $e^{0.71} \approx 2.03$
- $e^{4.96} \approx 142.59$
- $e^{2.84} \approx 17.12$

总和 ≈ 170.15

注意力权重：

- 我： $8.41 \div 170.15 \approx 0.05$
- 喜欢： $2.03 \div 170.15 \approx 0.01$
- 吃： $142.59 \div 170.15 \approx 0.84$
- 苹果： $17.12 \div 170.15 \approx 0.10$

“吃”独占了 84% 的权重。比前面朴素版的 64% 还要高。这是因为 QKV 投影之后，“吃”的分数（7）和其他词拉开了更大的差距，Softmax 进一步放大了这种差距。

第三步，对 Value 向量做加权求和。

注意：这里用的是 Value 向量，不是 Key 向量，也不是原始向量。Key 负责“被打分”，Value 负责“被取用”。这是两件不同的事。

四个词的 Value 向量：我 [0, 1]，喜欢 [1, -1]，吃 [1, 1]，苹果 [1, 0]。

$$\text{苹果的新向量} = 0.05 \times [0, 1] + 0.01 \times [1, -1] + 0.84 \times [1, 1] + 0.10 \times [1, 0]$$

逐个分量算：

- 第一维： $0.05 \times 0 + 0.01 \times 1 + 0.84 \times 1 + 0.10 \times 1 = 0 + 0.01 + 0.84 + 0.10 = \mathbf{0.95}$
- 第二维： $0.05 \times 1 + 0.01 \times (-1) + 0.84 \times 1 + 0.10 \times 0 = 0.05 - 0.01 + 0.84 + 0 = \mathbf{0.88}$

苹果的新向量：[0.95, 0.88]

“吃”的 Value 是 [1, 1]，它以 84% 的权重主导了加权求和。苹果的新向量被“吃”的信息深度灌注。

到这里，QKV 版本的注意力计算就完整地走完了。我们把每一步整理一下：

1. 每个词的原始向量分别乘以 W_Q 、 W_K 、 W_V ，得到三个投影向量 q 、 k 、 v
2. 用当前词的 q 和所有词的 k 做点积，得到原始分数
3. 每个分数除以 $\sqrt{d_k}$ （Query 和 Key 的维度的平方根）
4. Softmax 归一化，得到注意力权重
5. 用权重对所有词的 v 做加权求和，得到当前词的新向量

这五步就是一次完整的注意力计算。后面我们会看到，这五步可以被一个公式统一表达，而且可以用矩阵运算一口气算完所有词的结果。

8.2.6 一个看似不起眼却十分重要的除法

刚才的流程里，我们把 Q 和 K 的点积结果除以了 $\sqrt{d_k}$ 。在我们这个例子里 $d_k = 2$ ， $\sqrt{d_k}$ 只有 1.41，分数从 [3, 1, 7, 4] 变成了 [2.13, 0.71, 4.96, 2.84]，变化不算剧烈。你可能觉得这一步无关紧要。

但在真实的模型里， d_k 不是 2，而是 64、128 这样的数字。两个 64 维的向量做点积，是 64 个乘积加在一起。维度越高，累加的项越多，最终的数字就越容易变得很大。

问题是，如果点积的数值太大，送进 Softmax 之后会出大问题。我们用一组数字来看看。

顺着这个直觉，假设在一个高维空间里，三个词的点积分数分别是 [10, 3, 2]。这个量级在 $d_k = 64$ 的情况下很常见。

不做缩放，直接送进 Softmax：

$$e^{10} \approx 22026, e^3 \approx 20.1, e^2 \approx 7.4$$

$$\text{总和} \approx 22053.5$$

$$\text{权重: } [0.9988, 0.0009, 0.0003]$$

第一个词独占了 99.88% 的权重，另外两个词几乎为零。Softmax 退化成了“赢者通吃”：只选最大的那一个，其他全部抹掉。

这会导致两个后果。第一，加权平均变成了“只取一个词的 Value”，其他词的信息被完全忽略，注意力机制融合多个词的信息的能力消失了。第二，Softmax 在输出接近 0 或者接近 1 的区域，梯度几乎为零。梯度没了，参数更新不了，训练卡死了。

先除以 $\sqrt{d_k}$ 再做 Softmax：

$$d_k = 64, \sqrt{d_k} = 8. \text{ 缩放后的分数: } [10 \div 8, 3 \div 8, 2 \div 8] = [1.25, 0.375, 0.25]$$

$$e^{1.25} \approx 3.49, e^{0.375} \approx 1.45, e^{0.25} \approx 1.28$$

$$\text{总和} \approx 6.22$$

$$\text{权重: } [0.56, 0.23, 0.21]$$

第一个词仍然权重最高，但第二、三个词也各自拿到了超过两成。分布有明确偏好但不极端，既保留了注意力的选择性，又没有把信息通道完全堵死。

两组对比放在一起看：

	词 1	词 2	词 3
不缩放	0.999	0.001	0.000
除以 $\sqrt{d_k}$	0.56	0.23	0.21

图 8.5 把这个对比画了出来，差距是毁灭性的。不缩放的版本，注意力分布已经“硬”成了只看一个词，其他信息全部丧失；缩放之后，分布是软的，有偏好但不极端，梯度能正常流动，模型能继续学习。

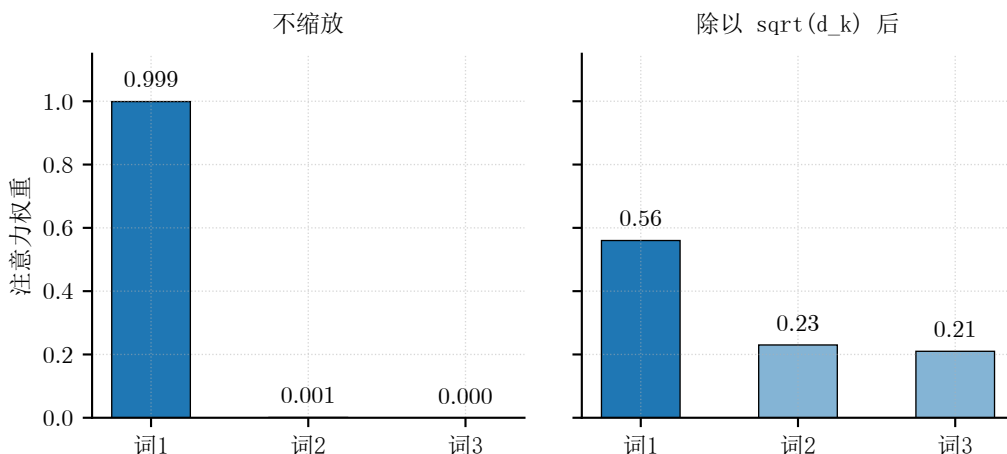


图 8.5: 缩放对 Softmax 输出的影响。左图不做缩放，99.9% 的权重集中在一个词上，其他词的信息被完全抹掉；右图除以 $\sqrt{d_k}$ 后，权重分布保持“有偏好但不极端”，每个词都能贡献信息。

实战避坑 · 缩放因子该除 d_k 还是 d_{model} ？一个字母写错，训练就会慢半拍

注意力分数要除以 $\sqrt{d_k}$ ， d_k 是 Query 和 Key 的维度。但在真实模型里， d_k 和 d_{model} 是两个不同的数。后面讲多头注意力的时候会看到， d_k 通常等于 d_{model} 除以头数，比如 $d_{model} = 4096$ 、32 个头， d_k 就是 128。有些新人自己实现注意力的时候，缩放因子写成了 $\sqrt{d_{model}}$ 而不是 $\sqrt{d_k}$ ，等于温度开大了好几倍，所有注意力分布都变得过于平均，模型学什么都慢半拍。排查方法：打印一下 Softmax 之前的分数的方差，如果方差明显偏小（大部分分数挤在 0 附近），很可能就是缩放因子算大了。

如果你还记得前面讲 Softmax 时提到的温度参数：温度越高，输出越均匀；温度越低，越集中在最大值。那么 $\sqrt{d_k}$ 在这里扮演的角色，本质上就是一个温度。它不是一个可有可无的工程小技巧，而是“没有就会崩”的必要操作。Transformer 的原始论文里专门解释了为什么要做这一步。

至于为什么偏偏是 $\sqrt{d_k}$ 而不是别的数，背后有一个统计上的道理。如果 Q 和 K 的每个分量都是均值为 0、方差为 1 的随机数，那么它们点积的方差恰好等于 d_k 。要把方差拉回 1，就需要除以标准差，也就是 $\sqrt{d_k}$ 。你不需要记住这个推导，只需要记住一个直觉：维度越高，点积越容易出大数字，除以 $\sqrt{d_k}$ 就是把这个膨胀效应中和掉。

8.2.7 为什么要拆成三个矩阵

到这里，你已经完整地看过了 Q、K、V 的计算流程。现在我们问一个更本质的问题：为什么非要用三个不同的矩阵？不能只用一个吗？

如果只用一个矩阵，每个词的原始向量经过同一个变换，得到一个新向量。然后用这个新向量既去打分，又被别人打分，又参与最终的加权求和。这比直接用原始向量好一点，至少向量经过了一次学出来的变换。但一个向量仍然在同时扮演三个角色，和我们在开头看到的问题本质相同。

三个矩阵的意义在于**角色分离**。

W_Q 让每个词生成一个专门用来“提问”的向量，也就是“我在找什么样的信息”。 W_K 让每个词生成一个专门用来“被查询”的向量，也就是“我能提供什么样的信息”。打分阶段只看 Query 和 Key 的匹配，跟 Value 没有关系。

这意味着，两个词在原始 Embedding 空间里可能长得完全不像（比如动词“吃”和名词“苹果”），但经过 W_Q 和 W_K 的投影之后，“苹果”的 Query 和“吃”的 Key 在新空间里高度匹配。**模型不是在算“谁跟我像”，而是在算“谁提供的信息最符合我的需求”**。我们在手算里看到的就是这个效果：原始点积里“苹果”给“吃”只打了 3 分（和给自己的 2 分差距不大），但 QK 点积里“苹果”给“吃”打了 7 分（甩开第二名 3 分）。同样的原始向量，经过不同的投影，匹配程度发生了质的变化。

然后是 Key 和 Value 的分离。这一点容易被忽视，但同样重要。

打分阶段用的是 Key，加权求和阶段用的是 Value。一个词“凭什么被找到”和“被找到之后它能给你什么”，是两件可以完全不同的事。回到我们的例子里看：“吃”的 Key 是 [3, 2]，它的 Value 是 [1, 1]。这两个向量完全不同。Key [3, 2] 的任务是让“吃”在打分环节被“苹果”的 Query [1, 2] 找到（两者点积为 7，全场最高）。Value [1, 1] 的任务是在“吃”被选中之后，把有用的信息内容传递给“苹果”。如果强制 Key 和 Value 用同一个向量，模型就得让这个向量同时在两件事上表现好：既要在打分环节拿高分，又要在取值环节提供合适的内容。这就像一个人既要负责在门口招揽客人，又要负责在后厨做菜，两件事抢同一个人，两件事都做不到最好。分开之后， W_K 专门优化“被找到”， W_V 专门优化“传递内容”，各自独立，各自做到最好。

注意力机制能学到东西，靠的就是这三个矩阵。没有它们，注意力就只是机械地算原始向量的相似度；有了它们，注意力变成了一个可以被训练的、灵活的信息检索系统。

面试考点 · 如果把 Q 和 K 用同一个矩阵投影，会怎样？

常见问法：“假设我令 $W_Q = W_K$ ，也就是 Query 和 Key 用同一个投影矩阵，注意力机制还能正常工作吗？会有什么后果？”

正确思路：能跑，但能力会打折扣。如果 $W_Q = W_K$ ，那么每个词的 Query 和 Key 是同一个向量，注意力分数矩阵 QK^T 就变成了 QQ^T 。这是一个对称矩阵，意味着词 A 对词 B 的分数和词 B 对词 A 的分数永远相等。注意力关系失去了方向性，模型没法学到“动词主动去找宾语、但宾语不一定主动去找动词”这种非对称的语义关系。此外， QQ^T 的对角线元素是每个向量和自己的点积（即向量长度的平方），通常会很大，导致每个词倾向于把最高的注意力给自己，而不是给上下文中真正重要的词。分开两个矩阵，就是为了让“提问”和“被查询”走不同的投影空间，打破这个对称性。

8.2.8 从逐个计算到矩阵一口气算完

前面的演示里，我们始终以“苹果”一个词为视角来走流程。但在实际运行中，句子里每个词都在同时做同样的事：拿自己的 Query 去和所有词的 Key 打分，然后用权重对所有词的 Value 加权求和。四个词就要各自走一遍完整流程。

好消息是，这四轮计算不需要一轮一轮串着做。它们可以被打包成一次矩阵运算。

做法是这样的：把四个词的 Query 向量竖着排成一个矩阵 Q （4 行 2 列），四个词的 Key 向量排成矩阵 K （4 行 2 列），Value 向量排成矩阵 V （4 行 2 列）。

然后计算 Q 乘以 K 的转置。转置你在学线性代数时见过，就是把矩阵的行和列互换。 K 原本是 4 行 2 列（每一行是一个词的 Key 向量），转置之后变成 2 行 4 列（每一列是一个词的 Key 向量）。为什么要转置？因为 Q 是 4 行 2 列， K 也是 4 行 2 列，两个矩阵直接乘是乘不了的，矩阵乘法要求前一个矩阵的列数等于后一个矩阵的行数。 K 转置之后变成 2 行 4 列， Q （4 行 2 列）乘以 K^T （2 行 4 列），前一个的列数 2 等于后一个的行数 2，可以相乘，结果是一个 4 行 4 列的矩阵。

这个 4×4 的矩阵，第 i 行第 j 列的数字，就是第 i 个词的 Query 和第 j 个词的 Key 的点积。也就是我们前面一个个手算的那些分数。一次矩阵乘法，16 个分数全部算完。

之后对这个 4×4 矩阵的每一行分别做 Softmax（每个词的注意力权重独立归一化），得到一个 4×4 的权重矩阵。然后用这个权重矩阵去乘 V 矩阵（4 行 2 列），也就是 4×4 乘以 4×2 ，结果是 4×2 ，恰好每一行就是一个词更新后的新向量。

面试考点 · Softmax 归一化的维度方向

常见问法：“注意力公式里的 Softmax，是对 QK^T 这个 $n \times n$ 矩阵的每一行做，还是每一列做？如果搞反了会怎样？”

典型错误：答不上来方向，或者说“应该无所谓吧，反正都是归一化”。

正确思路：是对每一行做 Softmax。 QK^T 的第 i 行存的是第 i 个词的 Query 和所有词的 Key 的点积，也就是第 i 个词对全句的注意力分数。对这一行做 Softmax，让这些分数变成合法的权重（加起来等于 1），然后用来对所有词的 Value 做加权平均。如果你搞反了，对每一列做 Softmax，归一化的就变成了“所有词对第 j 个词的关注度加起来等于 1”。这意味着你在约束“每个词被关注的总量是固定的”，而不是“每个词分配出去的关注总量是固定的”。语义完全变了，而且这样加权求和出来的结果不再是每个词独立地融合上下文，而是变成了一种全局的竞争机制。在代码里，这个 bug 就是 `dim=-1` 写成了 `dim=-2`（或者 `dim=0` 和 `dim=1` 搞反）。

整个过程用公式写出来：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

我们来对照这个公式，把每一块的 shape 理一遍。假设句子有 n 个词，Query 和 Key 的维度是 d_k ，Value 的维度也是 d_k （实际中也几乎都是这样设置的）：

- Q : $n \times d_k$ （ n 个词，每个词一个 d_k 维的 Query 向量）
- K^T : $d_k \times n$ （ K 转置之后）
- QK^T : $n \times n$ （所有词两两之间的分数）
- Softmax 后仍然是 $n \times n$ （每一行独立归一化）

- $V: n \times d_k$ (n 个词，每个词一个 d_k 维的 Value 向量)
- 最终结果: $n \times d_k$ (每个词一个更新后的 d_k 维向量)

实战避坑 · QK^T 那个 $n \times n$ 矩阵，才是显存杀手

注意力的计算量是序列长度的平方，但在工程实践里，真正先爆的不是计算时间，而是显存。 QK^T 会产生一个 $n \times n$ 的矩阵， n 是序列长度。一个 2048 token 的序列，这个矩阵有 400 万（2048 的平方）个元素；4096 token 就是 1600 万个。如果模型有 32 层、每层 32 个注意力头，那就是 $32 \times 32 = 1024$ 个这样的矩阵同时存在于显存中（因为反向传播需要保留它们来算梯度）。fp16 下，先按 batch size = 1 粗算，4096 长度的场景光是存这些分数矩阵就要吃掉 $1024 \times 4096^2 \times 2$ 字节 $\approx 32\text{GB}$ 显存，还没算 batch size 放大、模型参数和优化器状态。这就是为什么很多人把序列长度从 2048 翻倍到 4096 时，砍了一半 batch size，仍然 OOM。长序列、深层模型和半精度训练叠在一起时，QK 点积还可能出现极端值，接着把 Softmax 推成 inf 或 NaN。Flash Attention 之所以重要，不只是因为它算得快，更是因为它根本不需要把这个 $n \times n$ 矩阵完整地存下来，中间计算也更稳定。

这三步矩阵运算包括：Q 乘 K 转置、Softmax、再乘 V。它们可以整体交给矩阵运算处理，不需要按词逐个串行计算。GPU 天生就擅长做大规模矩阵乘法，这种计算方式正好把 GPU 的并行能力用到了极致。

最后一个容易混淆的地方。公式里的大写 Q、K、V，是整个句子所有词的向量堆叠成的矩阵，每一行对应一个词。而我们前面手算时用的“苹果的 Query”“吃的 Key”，是矩阵里的某一行，也就是单个词的向量。很多教程在公式和讲解之间来回切换时不说清楚这个区别，读者就容易在“这到底是一个向量还是一个矩阵”上卡住。记住：小写 q、k、v 是单个词的向量，大写 Q、K、V 是整句话所有词堆出来的矩阵。

比如这一节里的 Q 矩阵可以写成这样：

$$Q = \begin{bmatrix} q_{\text{我}} \\ q_{\text{喜欢}} \\ q_{\text{吃}} \\ q_{\text{苹果}} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \\ 1 & 3 \\ 1 & 2 \end{bmatrix}$$

第一行 $q_{\text{我}} = [0, 1]$ ，第二行 $q_{\text{喜欢}} = [1, 1]$ ，第三行 $q_{\text{吃}} = [1, 3]$ ，第四行 $q_{\text{苹果}} = [1, 2]$ 。K 和 V 也是同样的堆叠方式。

这个区分在读论文的时候影响理解，在写代码的时候直接影响对错。调试注意力模块时，shape 不匹配的报错十有八九和这个有关：你明明想取一个词的 query 向量出来做检查，结果传进去的是整个序列的 Q 矩阵，维度对不上，PyTorch 甩给你一条让人头大的错误信息。这种 bug 排查起来不难，但前提是你脑子里清楚大写和小写指的不一回事。

8.2.9 自检清单

1. Q 和 K 和 V 这三组向量，是来自三个不同的输入源吗？

不是。在自注意力中，它们全部来自同一组输入向量，只是分别乘以了三个不同的矩阵 W_Q 、 W_K 、 W_V 。同一个词、同一个原始向量，三次不同的投影。

2. 打分阶段用的是哪两组向量？加权求和阶段用的又是哪一组？
打分用的是 Query 和 Key 的点积。加权求和用的是 Value。Q-K 决定“关注谁”，V 决定“关注之后拿到什么”。
3. 如果去掉除以 $\sqrt{d_k}$ 这一步，会发生什么？
在高维空间里，点积的数值会很大，导致 Softmax 输出接近 one-hot（几乎全部概率集中在一个词上），梯度趋近于零，模型训练卡死。 $\sqrt{d_k}$ 缩放把点积的数值拉回合理范围，让 Softmax 保持“有区分度但不极端”的分布。
4. 公式里的大写 Q、K、V 和小写 q、k、v 有什么区别？
小写 q、k、v 是单个词的向量。大写 Q、K、V 是整个句子所有词的向量堆叠成的矩阵。公式 $\text{Attention}(Q, K, V)$ 表示的是对所有词同时计算注意力的矩阵运算。
5. W_Q 、 W_K 、 W_V 这三个矩阵是怎么来的？需要手动设计吗？
不需要。它们是模型的可训练参数，和线性层的权重一样，通过反向传播在训练过程中学出来。模型学到的不是“苹果应该关注吃”这种具体关系，而是一套通用的“如何打分”的能力。

8.2.10 Python 交互实验室

本节的手算过程涉及大量矩阵乘法和数值对比，纸面上能展示的组合终归有限。配套资源包中的 `qkv_attention_demo.py` 完整复现了上面所有计算步骤，并且在参数配置区标注了几组最值得动手尝试的改动。其中有一个实验特别推荐：把 W_Q 改成和 W_K 完全相同，运行后观察分数矩阵，你会发现它瞬间变成了对称矩阵，词 A 对词 B 的分数永远等于词 B 对词 A 的分数，注意力关系失去了方向性。这就是为什么 Query 和 Key 必须走不同的投影矩阵。另一个值得尝试的操作是把 W_V 改成单位矩阵（相当于取消 Value 投影），对比“苹果”最终的新向量发生了什么变化，那正是 Key 和 Value 为什么需要分开的直观证据。

8.3 多头注意力：为什么要从不同角度看同一句话

“我喜欢吃苹果”这句话里，“苹果”需要同时理解好几层关系。它是“吃”的对象，这告诉它自己是个食物。“我”是执行动作的人，有人在吃它。“喜欢”标记了情感色彩：不是被迫吃的，是心甘情愿地吃。

一个词要真正理解自己在句子中的角色，往往需要同时关注多个不同方向的信息。但注意力机制每次只产生一组分数。Softmax 归一化之后，四个词的注意力权重加起来恰好是 1，这意味着它们在互相抢配额。“苹果”给“吃”多分了一些权重，留给“我”和“喜欢”的就少了。如果“苹果”把大部分注意力集中在了“吃”身上，那留给“我”的注意力就所剩无几。

这就是单组 Q、K、V 的局限：一次注意力计算，只够一个词认真看一个方向。看了动词就顾不上主语，两者很难兼得。

怎么解决？思路并不复杂：不止做一次注意力计算，而是同时做好几次，每次用一组不同的投影矩阵，从不同的角度看同一句话，最后把所有角度的结果汇总起来。每一组独立的 Q、K、V 做一次完整的注意力计算，这一组就叫做一个头（head）。用多个头同时工作，就是多头注意力（Multi-Head Attention）。

Multi-Head Attention，翻译成中文是“多头注意力”。网上有人说，第一次看到这